Group 9: Vending Machine Simulator Design Document

Grewal Gaurav

Nwachukwu Obinna: 110195313

Mugunthan Vaaman

Elashaal Musa: 110180027

Aghbari Kareem: 110172122

Powers Casey: 110179162

Class: ELEC 2320 Engineering Software Fundamentals
Professor: Dr. Adewale

Problem Statement

A company is designing a new vending machine and requires an object-oriented software simulator to verify that the machine will meet the needs of both customers and service personnel. The simulator models a physical vending machine with a 10-row by 8-column grid of spring coils, each holding up to 10 items. It supports two operational modes: vending mode (door closed), where customers insert coins and purchase items, and restock mode (door open), where a service worker can set prices, refill coils, manage the cash and change boxes, and close the door. The machine accepts 5-cent, 10-cent, 25-cent, $1.00, and $2.00 coins, and rejects 50-cent coins.

The scope of this project includes building a console-based C++ simulator that supports both manual interaction (direct user commands) and an automatic simulation mode, where random customer and worker events are generated over time. The automatic mode uses a seeded pseudo-random number generator for reproducible results and tracks statistics to help answer operational questions, such as how much change should be loaded so the machine runs out of stock before it runs out of change.

System Architecture and Class Design

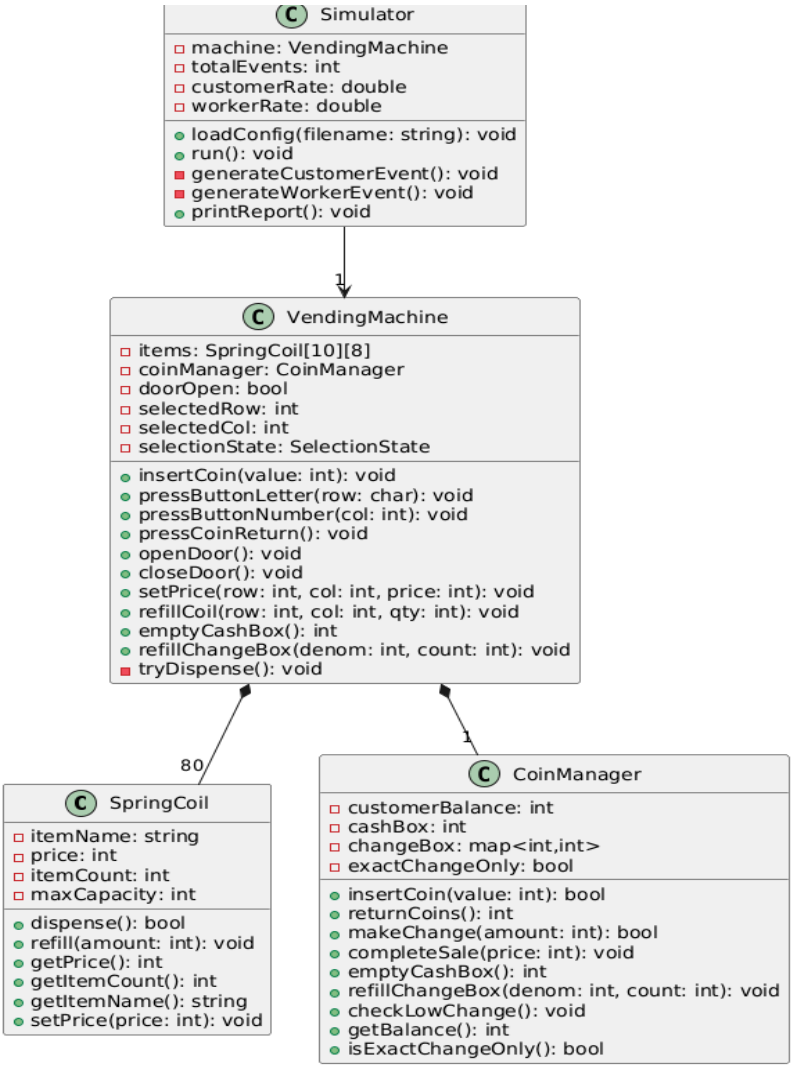
The system is built around four classes: VendingMachine, SpringCoil, CoinManager, and Simulator. Their relationships are shown in the UML diagram below.

VendingMachine is the central controller. It manages a 10×8 array of SpringCoil objects (rows A–J, columns 1–8) and a single CoinManager object. Customer selection is tracked through a SelectionState that progresses from no selection, to a row selection (A–J), to a column selection (1–8). The class maintains the current operating mode through a boolean doorOpen flag and vending mode when the door is closed and restock mode when the door is open. In vending mode, it delegates coin handling to CoinManager, records button presses, and calls tryDispense() to complete or reject a purchase. In restock mode, it exposes member functions to set prices, refill coils, empty the cash box, refill the change box, and close the door.

SpringCoil represents a single product slot. Each coil stores the product's name, price (in cents), current item count, and a maximum capacity of 10. Its dispense() member function decrements the count and returns success or failure, and refill() adds items up to the maximum. Prices are stored as integers in cents to avoid floating-point rounding errors.

CoinManager handles all monetary operations. It tracks the customer's running balance, maintains a change box (organized by denomination), and holds a cash box total. It provides member functions to insert a coin (rejecting 50-cent coins), return the customer's balance, make change using a greedy largest-denomination-first approach, complete a sale by transferring the price to the cash box, and check whether the change box is running low. When change is low, it sets an exactChangeOnly flag that the VendingMachine reads before completing transactions.

Simulator drives the automatic mode. It holds a reference to VendingMachine, loads configuration from a file, and uses configurable arrival rates to generate random customer and worker events. After each event it updates internal statistics counters. At the end of a run, it prints a summary report covering revenue, items dispensed, change usage, and error counts.



Requirements

Functional Requirements

The system must accept 5-cent, 10-cent, 25-cent, $1.00, and $2.00 coins and reject 50-cent coins, returning them through the coin return slot. It must display the customer's running total

after each accepted coin. It must allow the customer to select a row (A–J) and column (1–8) and attempt to dispense the corresponding item. When a selection is completed, the system must either dispense the item and return the correct change, display an "item unavailable" message if the coil is empty, or display an "insufficient funds" message if the balance is too low. The system must return all unspent coins when the coin return button is pressed. When the door is opened, the system must switch to restock mode, disabling all vending functions and allowing a service worker to set prices, refill coils, empty the cash box, refill the change box, and close the door. The system must display an "exact change only" warning when the change box falls below a defined threshold. The system must support an automatic simulation mode that generates random customer and worker events and collects statistics. The system must be able to print a statistics report on request.

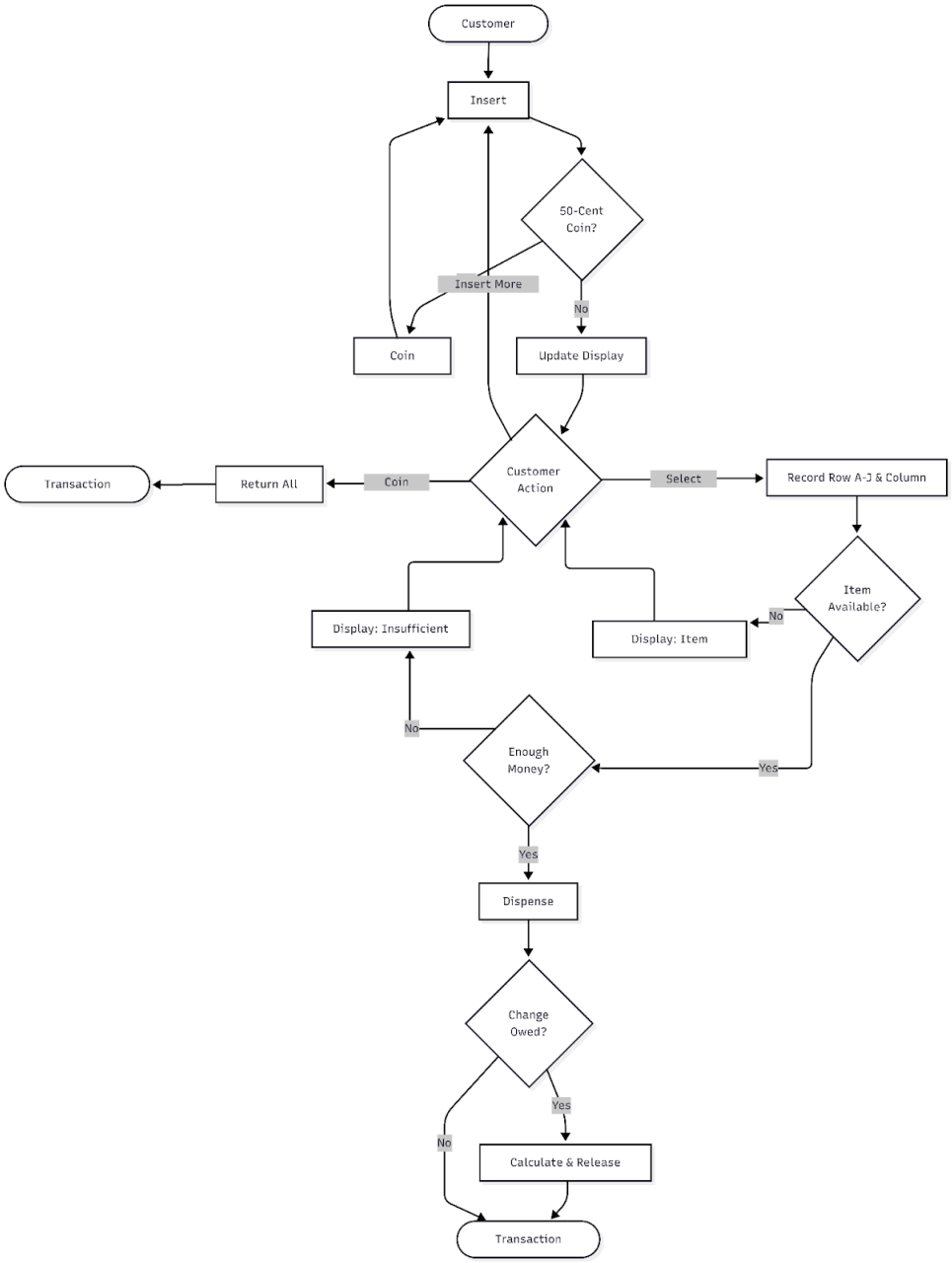
Non-Functional Requirements

The simulator must be implemented in C++ using object-oriented design principles. All monetary values must be stored as integers (cents) to ensure precision. The automatic simulation must produce reproducible results when given the same random seed. Console output must be clear and readable, with distinct prompts for vending and restock modes. The system must respond to each user action without noticeable delay. The codebase must be modular enough that the VendingMachine class operates identically whether driven by manual input or the Simulator.

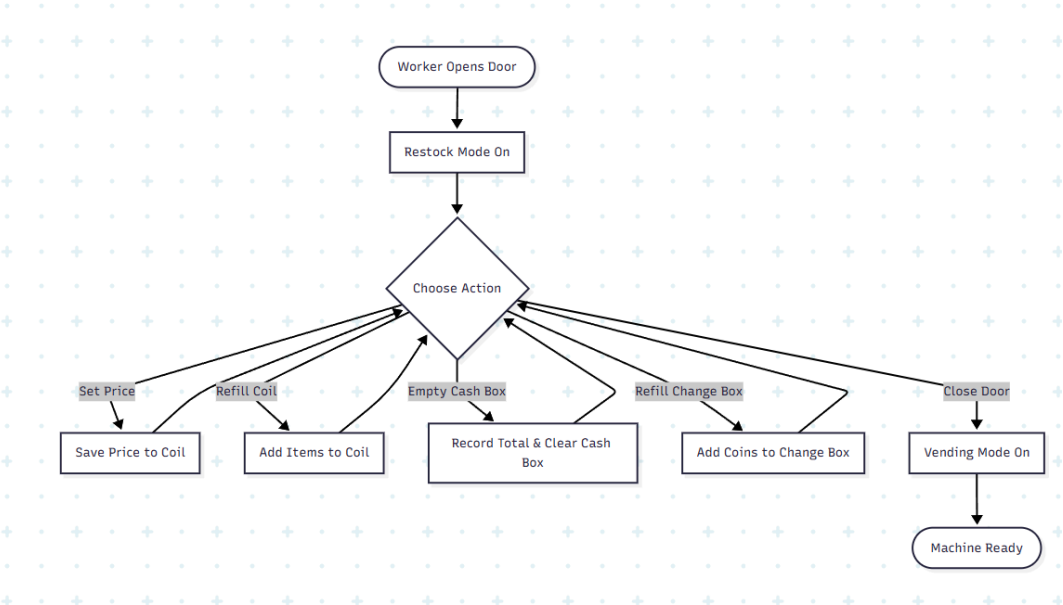
Flow Diagrams

The following three diagrams illustrate the main paths through the system.

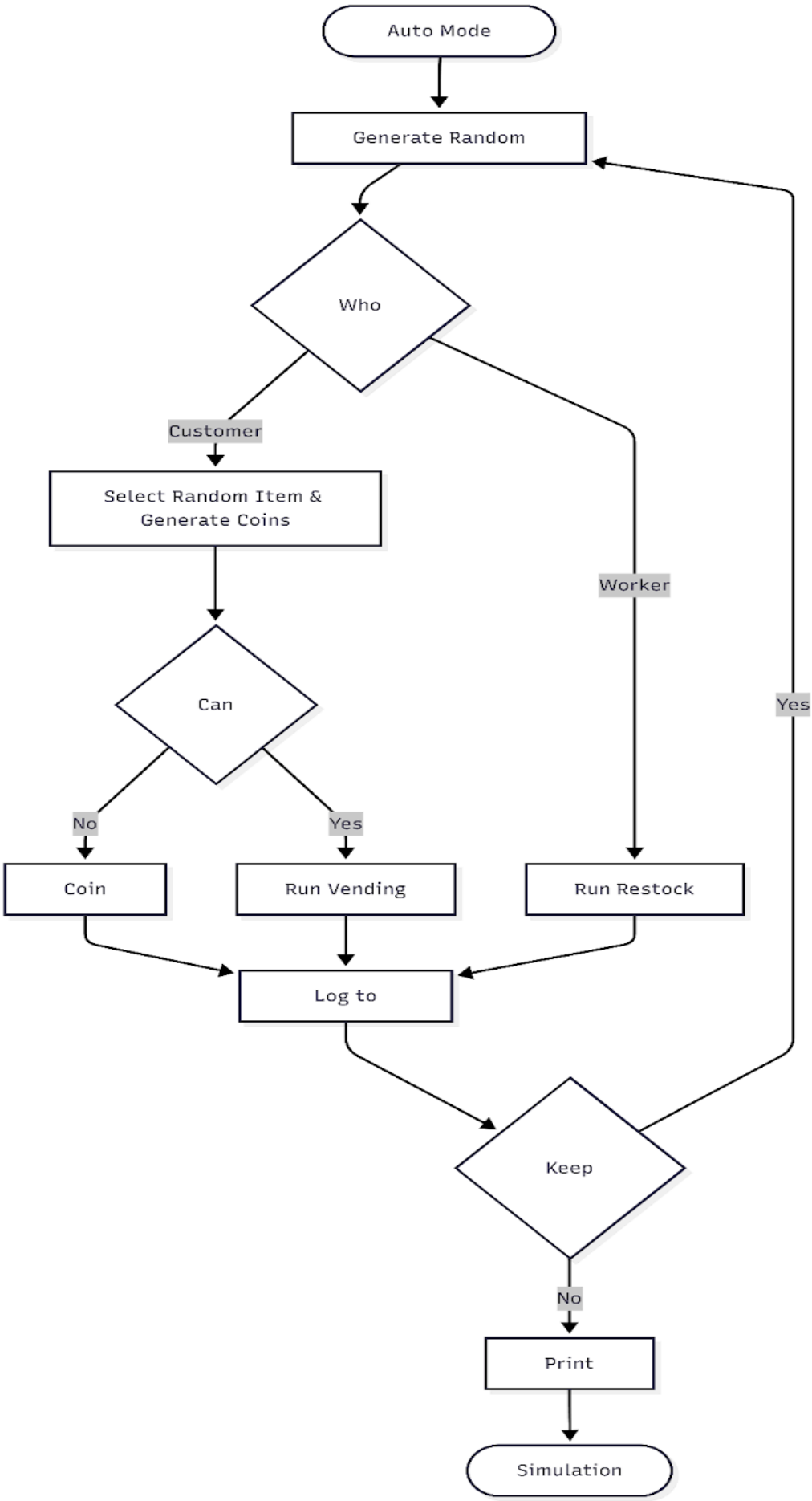
Customer Vending Flow (Fig. 1) traces a customer's interaction from arrival through coin insertion, item selection, and the three possible outcomes: successful purchase with change, an error message (item unavailable or insufficient funds), or cancellation via coin return.



Service Worker Restock Flow (Fig. 2) shows the sequence of actions available when the door is opened: setting prices, refilling coils, managing the cash and change boxes, and closing the door to return to vending mode.



Automatic Simulation Flow (Fig. 3) shows the simulation loop: generate an event, determine whether it is a customer or worker, run the appropriate flow, log statistics, and repeat until the simulation ends and a final report is printed.


Auto Mode
Generate Random
Who
Customer
Select Random Item & Generate Coins
Can
No
Yes
Coin
Run Vending
Worker
Run Restock
Log to
Keep
Yes
No
Print
Simulation

Algorithm Descriptions

Coin Insertion

When a coin is inserted, the machine checks whether the door is open (if so, the coin is rejected). Otherwise, if the coin is a 50-cent piece it is rejected via the coin return slot. For any other accepted denomination, its value is added to the customer's running total and the display is updated.

Item Selection and Dispensing

The customer presses a row button (A–J) then a column button (1–8). The machine checks the corresponding coil. If the coil is empty, it displays "item unavailable." If the customer's balance is less than the price, it displays "insufficient funds." Otherwise, the coil's item count is decremented by one and the machine proceeds to calculate change.

Change Calculation

The change owed is the difference between the customer's balance and the item's price. Starting from the largest denomination (200, 100, 25, 10, 5 cents), the machine dispenses as many coins as possible from the change box without exceeding the amount owed. Each coin is routed to the coin return slot. The customer's balance is then reset to zero.

Low Change Detection

After every transaction (and at startup), the machine sums the total value in the change box. If it falls below a defined threshold, an "exact change only" flag is set. While this flag is active, a transaction is only completed if the customer's balance exactly equals the item's price.

Coin Return

When the coin return button is pressed, all coins from the current transaction (not yet in the cash box) are routed to the coin return slot and the customer's balance is reset to zero.

Automatic Simulation

The simulator uses a random number generator to schedule customer and worker arrivals based on configurable average rates. A simulated customer randomly selects a coil, inserts a random combination of coins, and either completes the purchase or presses coin return if they cannot afford the item. A simulated worker opens the door, refills coils below a threshold, replenishes the change box, empties the cash box, and closes the door. After each event, statistics counters are updated. At the end, a summary report is printed.

Libraries and Tools

C++ is the required language for the project. It provides full support for object-oriented programming through classes, inheritance, and encapsulation.

<iostream> is the standard I/O library used for reading user input and printing display output to the console.

<string> is used for handling text such as display messages and error prompts.

<vector> is used for managing dynamic collections, such as coins in the change box.

<cstdlib> / <ctime> are used to seed and generate pseudo-random numbers for the automatic simulation mode.

<map> is used to associate coin denominations with their counts in the change box.

Git / GitHub is used for version control, enabling collaboration and change tracking among team members.

Task Distribution

Gauav will be responsible for the VendingMachine class. He will implement the main controller logic, door state management, selection state, and programming the logic that goes between SpringCoil and CoinManager.

Obinna will implement the CoinManager class. This includes coin insertion and rejection logic, the change box mappings, and the change making algorithm. He will also flag when exact change isn't possible.

Vaaman will implement the SpringCoil class and the overall project file structure, including dispense/refill logic, and price storage in integer cents.

Musa will handle the simulation class. He will build the automatic loop, configure arrival rates, the random number generator, and the statistics tracking system.

Kareem will be in charge of the console display layer. He will design the manual interaction prompts, display messages, and ensure output is clean and reusable.

Casey will handle integration, and ensure compatibility between users on the same GitHub repo. He will ensure both manual and automatic modes produce consistent results, and the workflow of the team remains steady.

Conclusion

This document has outlined the design of a vending machine simulator in C++, covering the problem statement, class structure, functional and non-functional requirements, interaction flows, algorithm logic, and planned tool usage. The simulator is self-contained, relying only on the C++ standard library. Both manual and automatic modes are supported through the same core machine logic, with the simulation layer operating independently on top of it. The system is well scoped for implementation and provides a solid foundation to build from as coding begins.